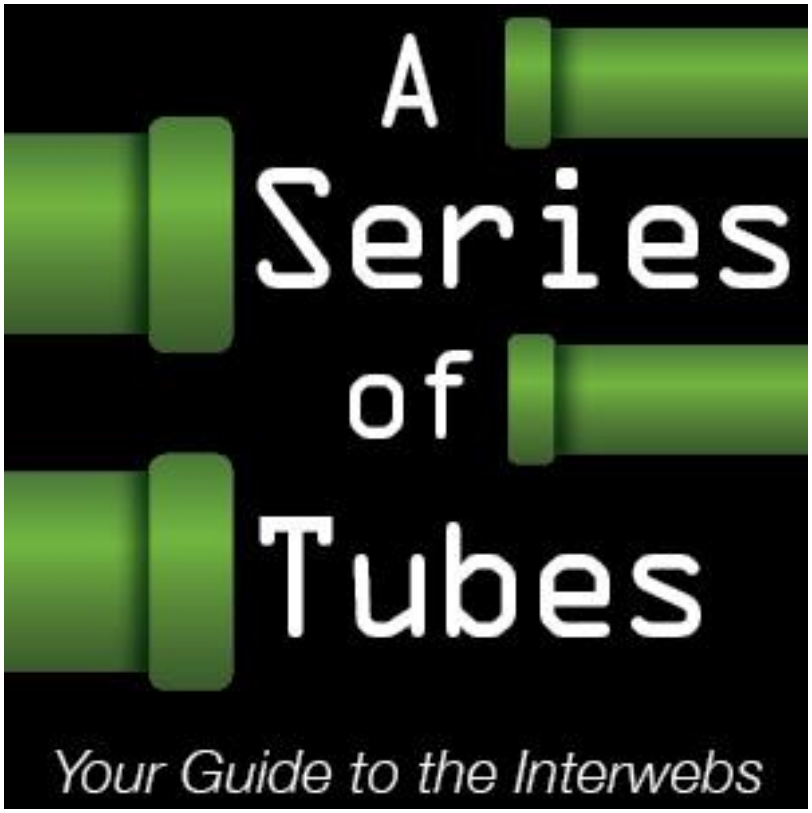
Static Pages



Agenda

- **Review**
- HTML
- CSS
- The request response cycle
  - URL encoding
  - HTTP

Review: how the web works

- The **client** asks the **server** for a page
- The **server** sends files back
- The **client's** browser displays the page

- This is called **request response cycle**.

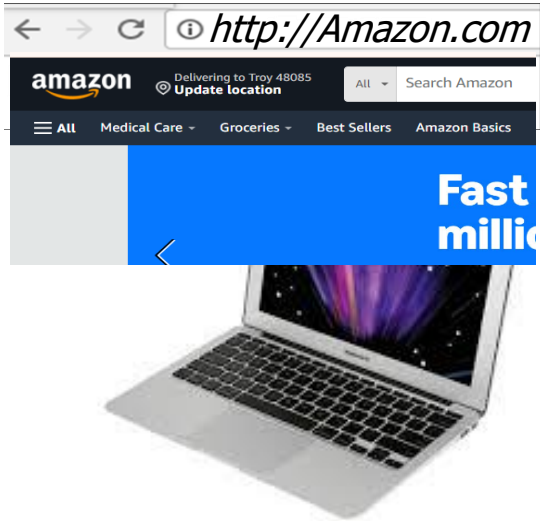
client

internet

server

Review: The request response cycle

Client: You



Client's browser renders the Files

Request: What's at amazon.com?

internet

Response: Files (HTML, CSS)

Server: amazon.com

Review: static pages

- A *static page doesn't change*
  - Often HTML,CSS, Images files
  - Every user sees the same page

```
<html lang="en">
  <body>
    Hello world!
  </body>
</html>
```

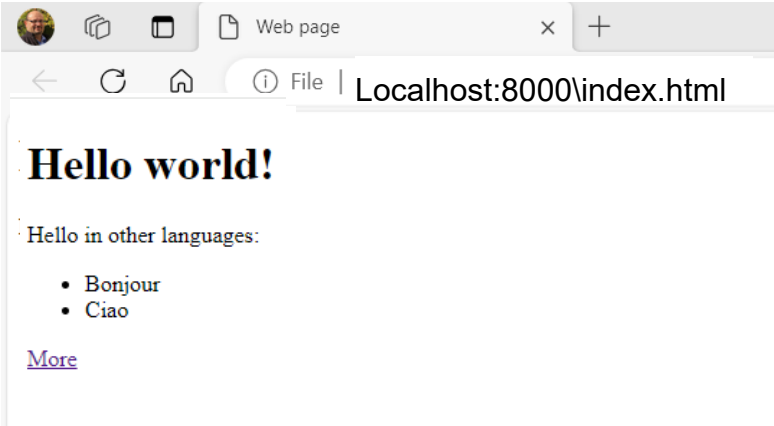
```
body {
    background: pink;
}
```

Agenda

- Review
- **HTML**
- CSS
- The request response cycle
  - URL encoding
  - HTTP

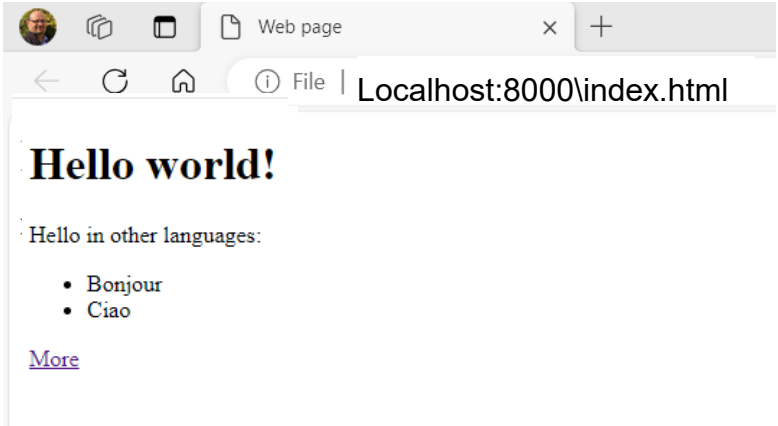
HTML: HyperText Markup Language

- How many tags are needed to create the following HTML page?



HTML: HyperText Markup Language

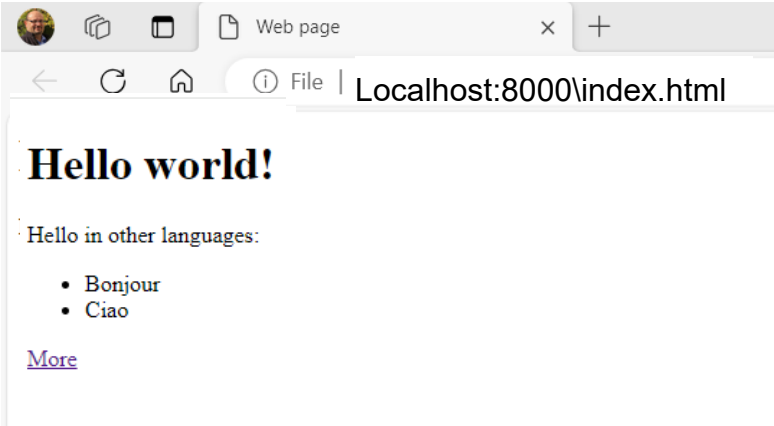
<!DOCTYPE html>



HTML: HyperText Markup Language

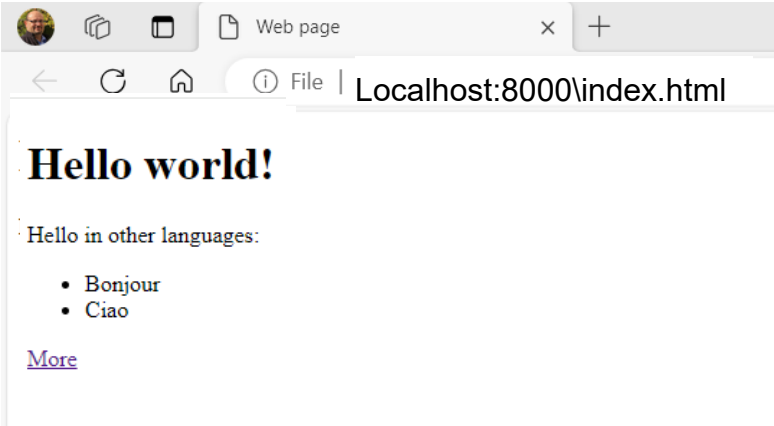
I'm html

<!DOCTYPE html>



HTML: HyperText Markup Language

```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
```

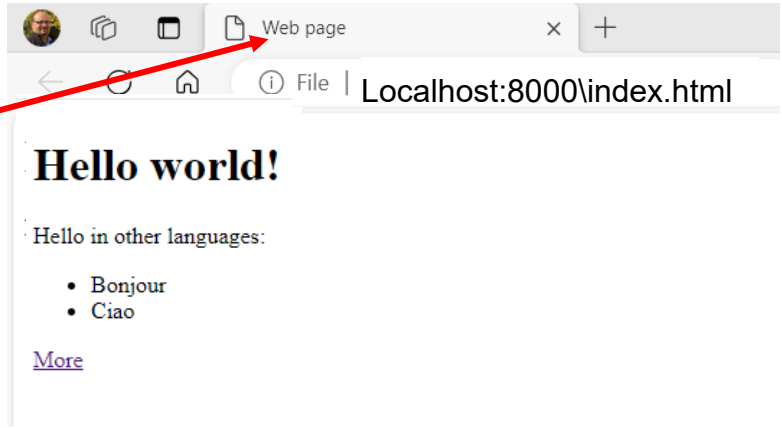


- HTML tags to determine structure and presentation; but only displays the content inside them.
- Not all HTML tags require closing tags, e.g.
 <img src="..">

HTML: HyperText Markup Language

```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
```

Title



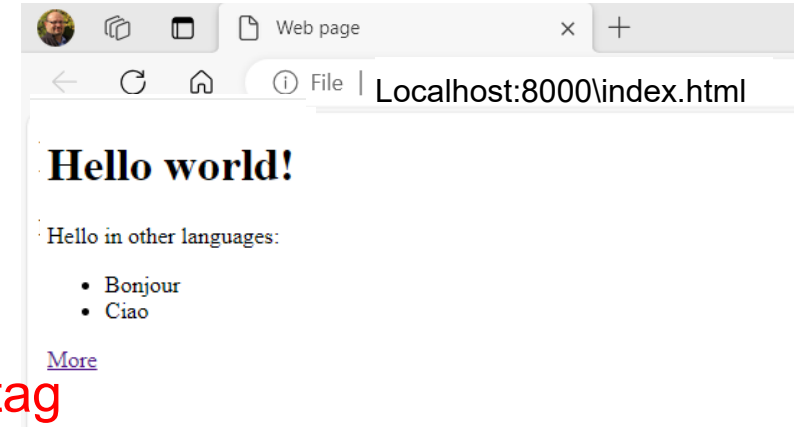
- HTML tags to determine structure and presentation; but only displays the content inside them.
- Not all HTML tags require closing tags, e.g.
 <img src="..">

HTML: HyperText Markup Language

```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
```

Open tag

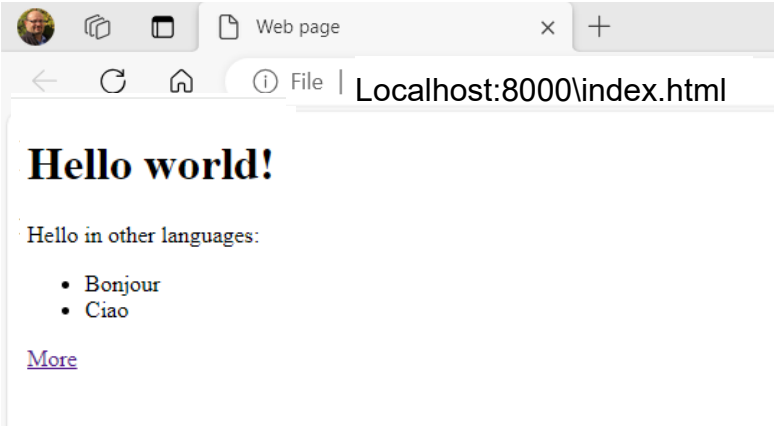
Close tag



- HTML tags to determine structure and presentation; but only displays the content inside them.
- Not all HTML tags require closing tags, e.g.
 <img src="..">

HTML: HyperText Markup Language

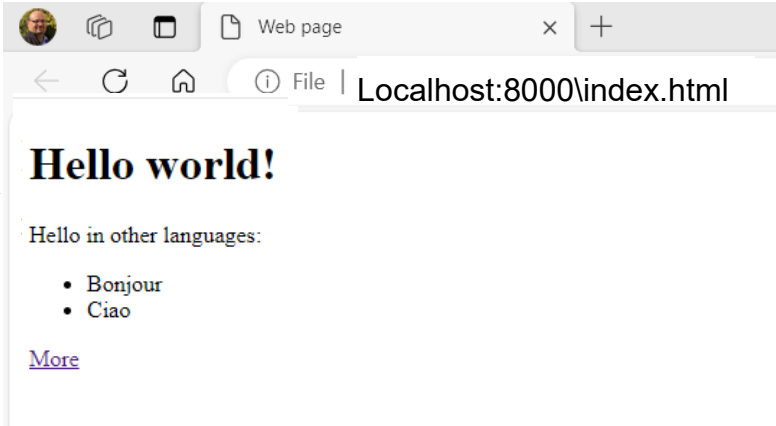
```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
<body>
    Hello world!
</body>
```



HTML: HyperText Markup Language

```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
<body>
    Hello world!
</body>
```

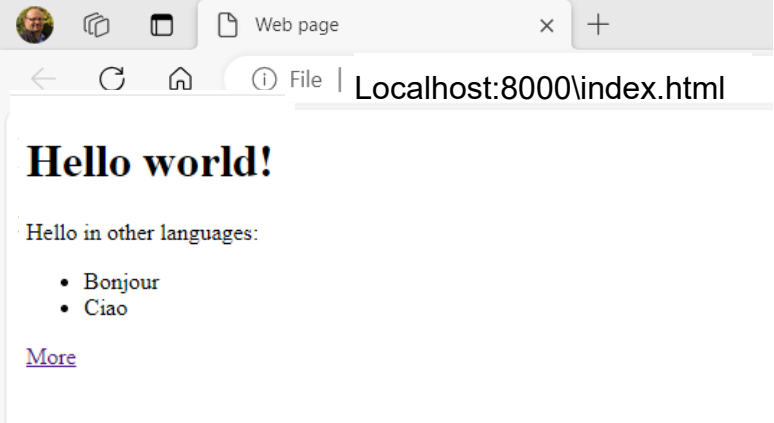
Body



HTML: <h1> through <h6>

- **<h1> … <h6>:** define headings.
  - <h1> represents the highest-level.
  - <h6> represents the lowest-level.

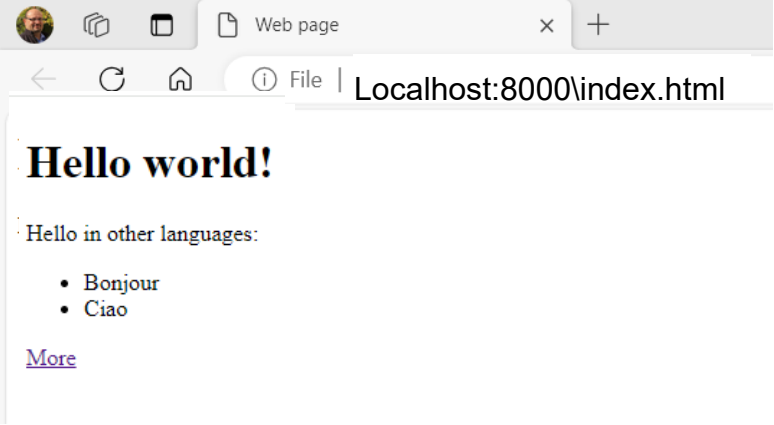
```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
<body>
    <h1>Hello world!</h1>
</body>
```



HTML: <p>

- **<p>:** Paragraph, start new line

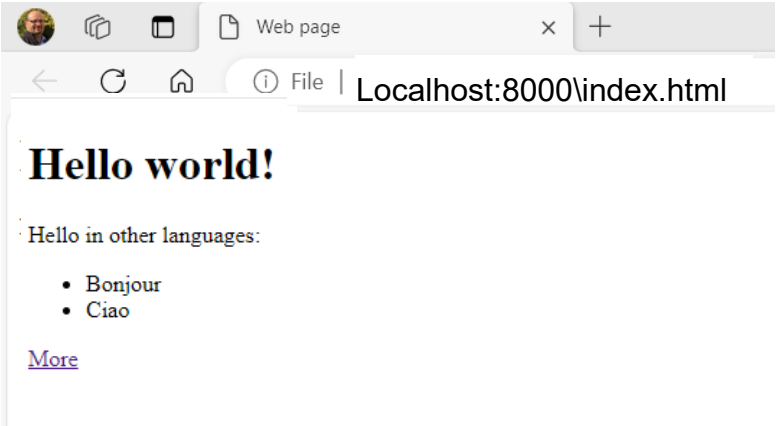
```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
<body>
     <h1>Hello world!</h1>
     <p> Hello in other languages: </p>
</body>
```



HTML: <ul> and <li>

<ul> and <li>: List of items.

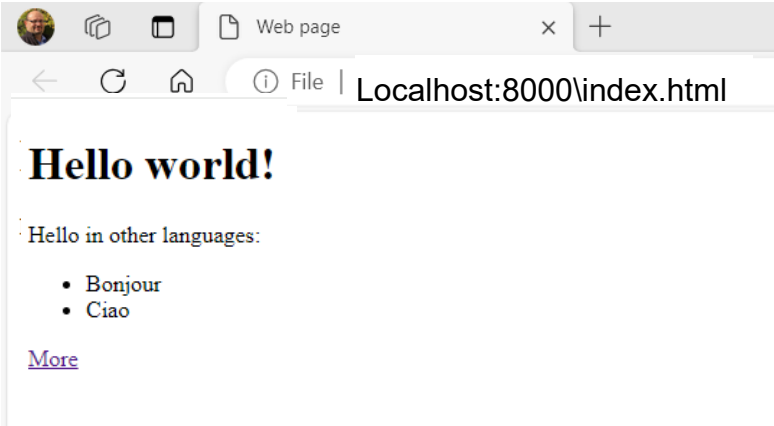
```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
<body>
     <h1>Hello world!</h1>
     <p> Hello in other languages: </p>
    <ul>
        <li>Bonjour</li>
        <li>Ciao</li>
     </ul>
</body>
```



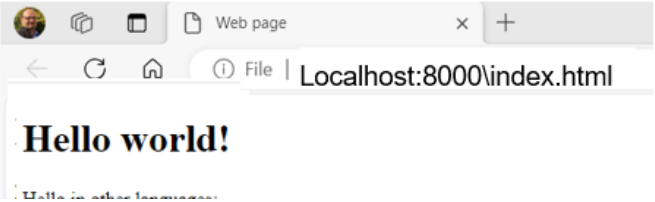
HTML: <ul> and <li>

<a> : create a hyperlink that navigates to another webpage.

```
<!DOCTYPE html>
<head>
    <title>Web page</title>
</head>
<body>
     <h1>Hello world!</h1>
     <p> Hello in other languages: </p>
    <ul>
        <li>Bonjour</li>
        <li>Ciao</li>
     </ul>
     <a href="index2.html"> More </a>
</body>
```

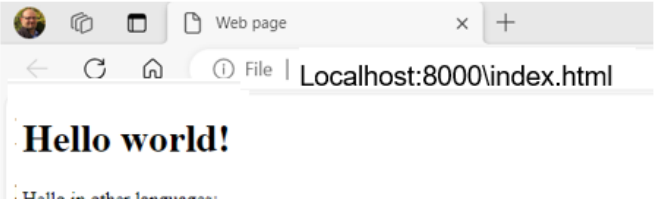


Escape strings in HTML



- Some characters have special meaning in an application context
  - Example: “<“ in HTML
- What if you want to communicate such a character as an ordinary character?
  - Example: “I < 3 chickens”
- Need an escape string
  - Example: “I **<** 3 chickens”

Escape strings in URLS

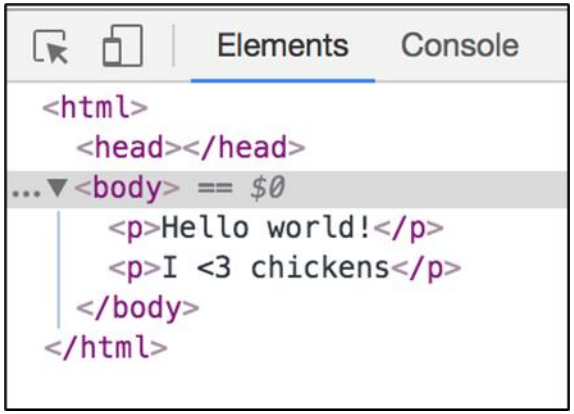


- Some characters have special meaning in an application context
  - Example: “ “ (a space) in a URL
- What if you want to communicate such a character as an ordinary character?
  - Example: hello world.jpg
- Need an escape string
  - Example: hello**%20**world.jpg

Document Object Model (DOM)

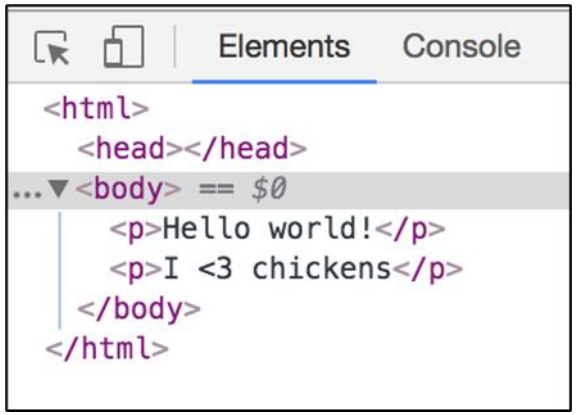
- In addition to rendering the webpage, the browser will create the DOM.
- HTML tags form a tree that is called the Document Object Model (DOM).

```
<html>
  <head></head>
  <body>
    <p>Hello world!</p>
    <p>I &lt;3 chickens</p>
  </body>
</html>
```



Document Object Model (DOM)

- Inspect the DOM with
  - Chrome developer tools
  - Firefox developer tools



Agenda

- Review
- HTML
- **CSS**
- The request response cycle
  - URL encoding
  - HTTP

CSS

- Cascading Style Sheets
- Add style to web pages:
  - Colors
  - Layout

CSS – how to link an HTML with a CSS file.

- HTML for content

```
<!-- index.html -->
<html>
  <head>
  </head>
  <body>
    <p>Hello world!</p>
  </body>
</html>
```

CSS – how to link an HTML with a CSS file.

- HTML for content

```
<!-- index.html -->
<html>
  <head>
    <link rel="stylesheet" type="text/css" href="style.css">
  </head>
  <body>
    <p>Hello world!</p>
   </body>
</html>
```

CSS – how to link an HTML with a CSS file.

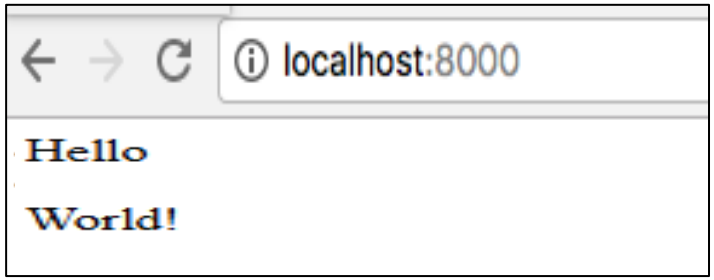
- CSS for layout

```
/* style.css */
body {
    background-color: lightblue;
}
p {
    margin-left: 20px;
}
```

HTML: Example

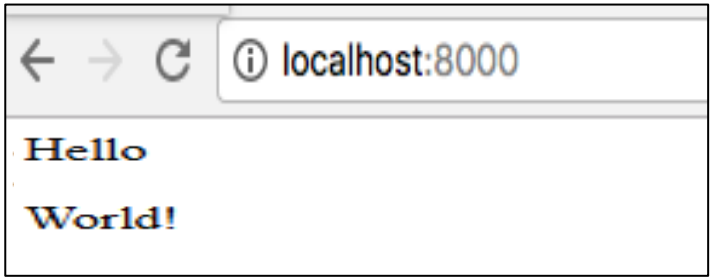
- Color each <p> element in a different color?

```
<html lang="en">
  <body>
    <p>Hello</p>

    <p>World!</p>
  </body>
</html>
```



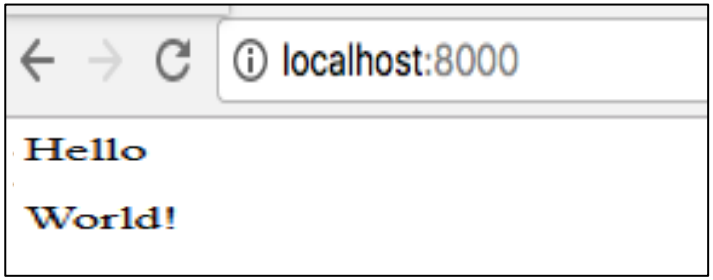
HTML: Example

```
<html lang="en">
  <body>
    <p class="first">Hello</p>

    <p class="second">World! </p>
  </body>
</html>
```



HTML: Example

```
<html lang="en">
  <body>
    <p class="first">Hello</p>
    <p class="second">World!</p>
  </body>
</html>
```



CSS: Example

style.css

```
.first {
   color: red;
}
.second {
   color: blue;
}
```

CSS: Example

```
style.css
    .first {
        color: red;
    }
    .second {
        color: blue;
    }
```

```
<html lang="en">
  <head>
    <link rel="stylesheet" type="text/css" href="/style.css">
  </head>
  <body>
    <p class="first">Hello</p>
    <p class="second">world!</p>
  </body>
</html>
```

Agenda

- Review
- HTML
- CSS
- **The request response cycle**
  - **URL encoding**
  - HTTP

Uniform Resource Locator (URL)

- The client requests a web page by typing “https://en.wikipedia.org/wiki/URL" in the browser's address bar

URL: Protocol

- https://en.wikipedia.org/wiki/URL

- Protocol: what "language" is used for request-response cycle

- HTTP
- HTTPs – Secure version of HTTP

URL: Server

- https://en.wikipedia.org/wiki/URL

- Which computer we are communicating with
- localhost or 127.0.0.1 means the same computer as the web browser

URL: Port

- https://en.wikipedia.org:443/wiki/URL

- More than one program is running on the server computer, and the port number specifies which program to communicate with.

- Port can be implied
  - HTTP: 80
  - HTTPS: 443

URL: Port

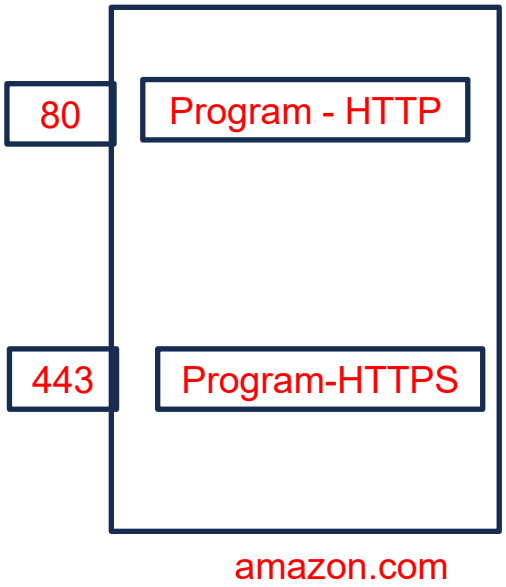
- https://en.wikipedia.org:443/wiki/URL

- More than one program is running on the server computer, and the port number specifies which program to communicate with.

- Port can be implied
  - HTTP: 80
  - HTTPS: 443

http://amazon.com:80

https://amazon.com:443



URL: Path

- https://en.wikipedia.org/wiki/URL

- Like a file path
- If no file is specified, default is index.html

- http://localhost:8000/ is the same as http://localhost:8000/index.html

Absolute vs. Relative paths

- **Absolute path**
  - Always starts from the root directory (/) of the website.
  - Starts with a slash (/) – root directory.

  - Examples: `<img src="/uploads/image.jpg">`

  - No matter where the current page is:

  /index.html
  /users/profile.html
  /admin/settings.html

  - The browser always goes to (/uploads/image.jpg)

Absolute vs. Relative paths

- **Relative path**
  - The location of a file based on the directory you are currently in.
  - Doesn't start with a slash (/)

  - Example, if current page is:

    `/users/awdeorio/profile.html`

  And you write:

    `<img src="uploads/image.jpg">`

  Browser goes to:

    `/users/awdeorio/uploads/image.jpg`

URL: Query

https://en.wikipedia.org/w/index.html?title=ABC&action=Print

- Parameters sent to the server inside the URL
- Key-value form

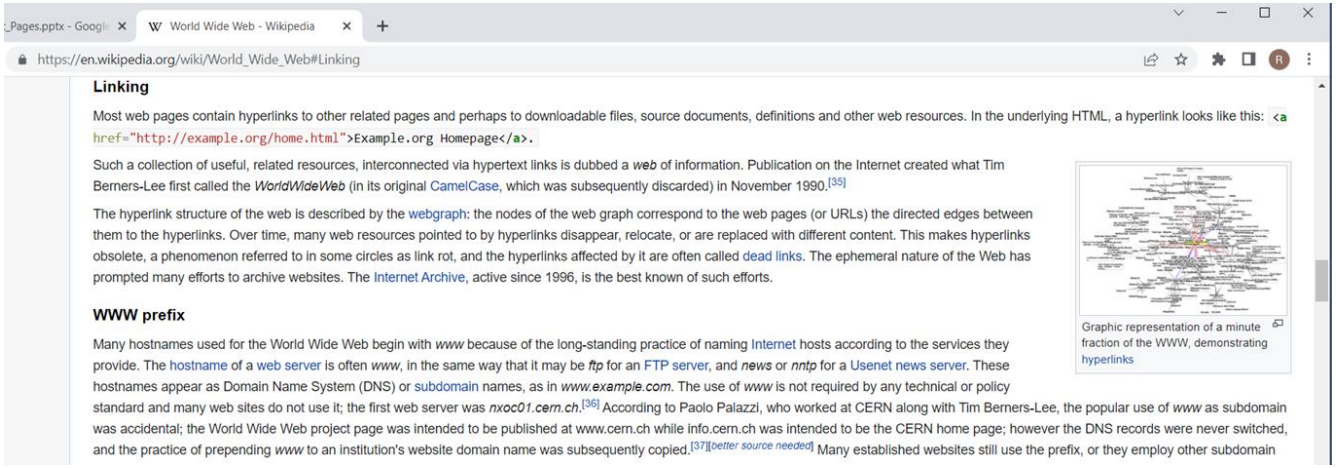
- We won't use these much in 485

URL: Fragment

https://en.wikipedia.org/wiki/URL#fragment

- Only used by web browser, not server
- We won't use these much in 485
- Example: navigate directly to the section labeled "Linking"

http://en.wikipedia.org/wiki/World_Wide_Web**#Linking**



Agenda

- Review
- HTML
- CSS
- The request response cycle
  - URL encoding
  - **HTTP**

Details of the Request/ Response

```
> GET / HTTP/1.1
> Host: cse.engin.umich.edu
> User-Agent: Chrome/7.54.0
> Accept: */*
```

request

```
< HTTP/1.1 200 OK
< Date: Tue, 12 Sep 2017 20:04:20 GMT
< Server: Apache/2.2.15 (Red Hat)
< Accept-Ranges: bytes
< Connection: close
< Transfer-Encoding: chunked
< Content-Type: text/html; charset=UTF-8
```

response

Details of the Request

```
> GET / HTTP/1.1
> Host: cse.engin.umich.edu
> User-Agent: Chrome/7.54.0
> Accept: */*
>
```

- Type of HTTP request methods:
  - **GET**: Ask for data.
    Example: Open a webpage
  - **HEAD**: Ask for page info only (no content)
    Example: Check if a page has changed
  - **POST**: Send data to the server
    Example: Submit a login form

Details of the Request

```
> GET / HTTP/1.1
> Host: cse.engin.umich.edu
> User-Agent: Chrome/7.54.0
> Accept: */*
>
```

- The path portion of the URL being requested.
- Slash (/) means the main page/homepage of the server.

HTTP: Hypertext Transfer Protocol

```
➢ GET / HTTP/1.1
  > Host: cse.engin.umich.edu
  > User-Agent: Chrome/7.54.0
  > Accept: */*
  >
```

- Request/response protocol
  - Client (your browser) opens connection to server and writes a request
  - Server responds appropriately
  - Connection is closed
  - That’s it

- Server can't open connection to client.
- **HTTP is stateless**: the server does not remember previous requests from the same client.

Details of the Request

```
> GET / HTTP/1.1
> Host: cse.engin.umich.edu
> User-Agent: Chrome/7.54.0
> Accept: */*
>
```

- Which server we are communicating with
  cse.engin.umich.edu.

Details of the Request

```
> GET / HTTP/1.1
> Host: cse.engin.umich.edu
> User-Agent: Chrome/7.54.0
> Accept: */*
>
```

- Which browser is making the request. E.g. Chrome/Safari

Details of the Request

```
> GET / HTTP/1.1
> Host: cse.engin.umich.edu
> User-Agent: Chrome/7.54.0
> Accept: */*
>
```

- Which file types the client will accept.

Details of the Response

```
< HTTP/1.1 200 OK
< Date: Tue, 12 Sep 2017 20:04:20 GMT
< Server: Apache/2.2.15 (Red Hat)
< Accept-Ranges: bytes
< Connection: close
< Transfer-Encoding: chunked
< Content-Type: text/html; charset=UTF-8
```

Details of the Response – Status Code

```
< HTTP/1.1 200 OK
< Date: Tue, 12 Sep 2017 20:04:20 GMT
< Server: Apache/2.2.15 (Red Hat)
< Accept-Ranges: bytes
< Connection: close
< Transfer-Encoding: chunked
< Content-Type: text/html; charset=UTF-8
```

- Status Code meanings:
  - 1XX: Informational
  - 2XX: Successful, e.g. 200 OK
  - 3XX: Redirection error
  - 4XX: Client error, e.g. 404 Not Found
  - 500: Server error

Details of the Response: Headers

```
< HTTP/1.1 200 OK
< Date: Tue, 12 Sep 2017 20:04:20 GMT
< Server: Apache/2.2.15 (Red Hat)
< Accept-Ranges: bytes
< Connection: close
< Transfer-Encoding: chunked
< Content-Type: text/html; charset=UTF-8
```

- Most are optional and describe how the data will be transferred

Details of the Response: Content-Type

```
< HTTP/1.1 200 OK
< Date: Tue, 12 Sep 2017 20:04:20 GMT
< Server: Apache/2.2.15 (Red Hat)
< Accept-Ranges: bytes
< Connection: close
< Transfer-Encoding: chunked
< Content-Type: text/html; charset=UTF-8
```

- Content-type: what kind of file it is
  - text/html
  - image/png

- Character encoding
  - UTF-8 most common – it's like ASCII from C++ but with support for non-English languages

Summary

- A *static page* produces the same content every time
  - Often just HTML/CSS
- HTML is for data presentation
  - How does it look?
- CSS defines presentation.
- HTTP is a request/response detail.
  1. Client opens connection to server and writes a request
  2. Server responds
  3. Connection is closed